

Performance Test

Test Run ID: 3ed52948-0aad-4938-bafe-d4ce81f31d14

This chapter provides a summary of the test run metrics. The tables contain the aggregated values of the metrics for the entire test run.

Date	05 June 2026	Max VUs Tested	60
Executor	ramping-vus	Tool	k6

1. Executive Summary

The target endpoint for this performance test is the "https://quickpizza.grafana.com" URL. The testing type is a spike test, which involves a rapid increase in load to simulate a sudden surge in traffic. The load profile consists of three stages: an initial ramp-up to 60 Virtual Users (VUs) over 5 seconds, followed by a sustained load of 60 VUs for 15 seconds, and then a gradual decrease to 10 VUs over 10 seconds, with a final 10-second ramp-down period.

Stable Threshold	Degradation Onset	Critical Point	Peak Throughput
1-60 TPS	N/A	N/A	60 TPS

2. Performance Summary

Trends

metric	avg	max	med	min	p90	p95	p99
http_req_duration	244.53ms	875.96ms	230.35ms	217.07ms	243.63ms	251.38ms	-
http_req_waiting	242.31ms	875.72ms	229.72ms	215.63ms	242.92ms	247.78ms	-
http_req_connecting	14.45ms	772.95ms	0.00ms	0.00ms	0.00ms	221.63ms	-
iteration_duration	1275.06ms	2592.52ms	1232.33ms	1218.02ms	1250.84ms	1688.99ms	-

Counters

metric	count	rate
data_received	3.92 MB	128.81 kB/s
data_sent	161.33 kB	5.18 kB/s
http_reqs	1,139	36.55/s
iterations	1,139	36.55/s

Rates

metric	rate
checks_succeeded	100.00% (1139/1139)
checks_failed	0.00% (0/1139)
http_req_failed	0.00% (0/1139)

Gauges

metric	value
vus	3 (min: 3, max: 60)
vus_max	60 (min: 60, max: 60)

Checks

Check Name	Passed	Failed	Success Rate	Status
status is 200	1139	0	100.00%	Pass

3. Advanced Analysis

3.1 Error Analysis

- No error or warning logs found in the test execution.
- No metrics abnormalities, error rate spikes, or failed checks detected.

3.2 Observation

The performance test results indicate a successful execution with no errors or warnings. However, upon closer inspection of the metrics, we notice that the average duration of HTTP requests (``http_req_duration``) is relatively high, with an average value of 244.53ms. This could be due to various factors such as:

- **Client timeouts:** The ``http_req_duration`` metric includes the time it takes for the client to establish a connection, send the request, and receive the response. If the client is experiencing timeouts, it may lead to increased average durations.
- **Server-side processing:** The high average duration could also indicate that the server is taking a significant amount of time to process the requests. This could be due to various factors such as database queries, complex business logic, or resource-intensive operations.
- **Network latency:** The ``http_req_connecting`` and ``http_req_tls_handshaking`` metrics indicate that the client is spending a significant amount of time establishing a connection and performing TLS handshakes. This could be due to network latency or issues with the client's connection to the server.

3.3 Final Recommendations

1. **Advanced Connection Management:** Implement a connection pooling mechanism to reduce the overhead of establishing new connections for each request. This can be achieved using libraries such as ``pgbouncer`` for PostgreSQL or ``mysql-connector-java`` for MySQL.
2. **Session Persistence sticky affinity/Redis cache:** Implement session persistence using sticky affinity or Redis caching to reduce the load on the server and improve response times.
3. **Gateway resilience Istio outlier circuit breaker:** Implement an Istio outlier detection circuit breaker to detect and prevent cascading failures in the gateway.
4. **Observability tracing and standardized log levels:** Implement observability tracing using tools such as OpenTracing or Jaeger to gain insights into the request flow and identify bottlenecks. Standardize log levels to ensure consistent logging across the application.
5. **Database read/write splitting and PgBouncer connection pooling:** Implement database read/write splitting to reduce the load on the database and improve response times. Use PgBouncer or similar connection pooling mechanisms to reduce the overhead of establishing new connections for each request.

3.4 Conclusion

The performance test results indicate a successful execution with no errors or warnings. However, the high average duration of HTTP requests suggests that there may be opportunities for optimization. Implementing the recommended changes should help improve response times and reduce the load on the server.